

Machine Learning Applications

AI-Based Surface Defect Detection and Context-Aware Inspection Assistant Using YOLOv8 and Retrieval-Augmented Generation (RAG)

Submitted By

C.T Lahiru Pramuditha

Index Number: Cobseds252P-027

Acknowledgement

Thank you to all who helped me with this Machine Learning Applications Coursework. I would like to thank my module lecturer for his excellent advice, feedback and support throughout the process of this project. Their knowledge and direction was a big help in the completion of this work.

I am also grateful to the Department of Computing and the University for the learning resources and facilities provided. Last but not least, I would like to express my gratitude to the members of my family and friends for their encouragement and support during this project.

Abstract

Surface defects detection is one of the significant works in structural inspection because it can detect the visible defect like cracks, holes etc. as early as possible. This project suggests an automated defect detection system, which is time-consuming and subjective to be inspected manually. The manual inspection is often time-consuming and subjective, so this project proposes an automated defect detection system using the YOLOv8 object detection model.

Three classes (Normal, Crack, Hole) were prepared, preprocessed and annotated according to the YOLO format, which is a custom dataset. The dataset was partitioned into training, validation, and test sets, and the YOLOv8 model was trained for 100 epochs. The dataset was split into 100 epochs of training, validation and testing. Normalized object detection metrics were used to assess the trained model, such as precision, recall, F1-score and mean Average Precision (mAP). The qualitative results were also obtained by selecting representative prediction examples of unseen test images.

For ease of use, the trained model was embedded into a web application using Streamlit that enabled users to upload an image, get automatic defect detection, visualize the results of the prediction, and can interact with an AI-powered chatbot for further instructions.

The experimental results prove that the proposed system is able to precisely detect and locate cracks and holes, and correctly identify normal surfaces under most testing situations. While there are still some constraints in the visualization of similar surface patterns, the prototype

developed in this work shows the feasibility of deep learning for automated surface defect inspection, and will form the basis for future work to enhance it.

Table of Contents

Acknowledgement	1
Abstract	2
1 Introduction.....	4
1.1 System Overview	4
2. System Design and Technology Selection.....	5
2.1 System Architecture	5
2.2 Technology Justification	7
3 Computer Vision Model Development	8
3.1 Dataset Description	8
3.2 Data Collection and Class Verification.....	9
3.3 Data Pre-processing and Annotation.....	11
3.4 YOLO Model Development and Training	12
3.5 Model Outputs and Training-Stage Analysis	13
4 Model Evaluation and Performance Analysis.....	16
4.1 Evaluation Approach.....	16
4.2 Training and Validation Performance	16
4.3 Analysis of precision, recall and F1-score.	16
4.4 Detection Examples and Qualitative Evaluation.....	17
4.5 Error Analysis and Model Limitations.....	19
4.6 Evaluation Summary	19
5 Conclusion and Future Directions	20
5.1 Conclusion.....	20
5.2 Limitations	20
5.3 Future Work	21
REFERENCES	22
Appendix Part-A	22
Appendix Part-B	25

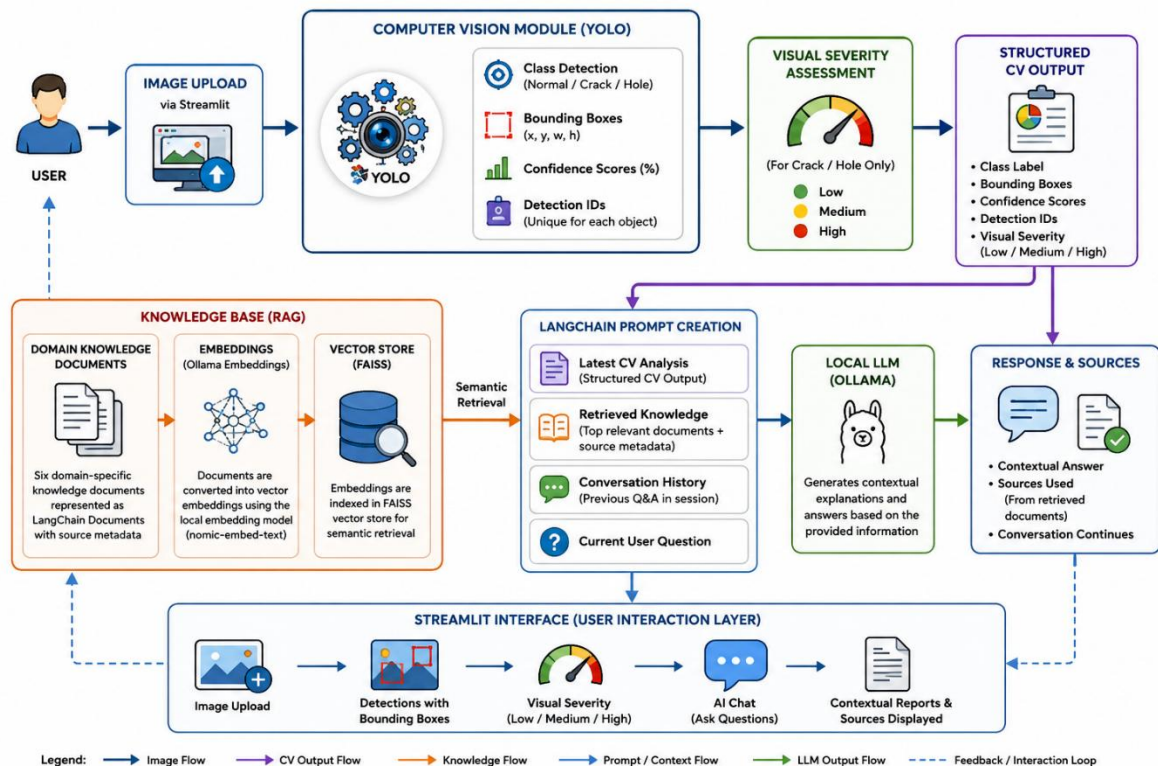
1 Introduction

1.1 System Overview

This project proposes a prototype of a computer vision based surface defect inspection system with the help of the Large Language Model (LLM), Retrieval-Augmented Generation (RAG) and an interactive application. The system aims to analyse the uploaded surface image and determine whether the conditions are Normal, Crack or Hole, and to give context information for further investigation and interpretation.

The computer vision component is based on a model developed by YOLO for the identification of objects contained in a photo, which is uploaded to the computer. The application displays a bounding box, class prediction, confidence score and a unique detection identifier for each detected region. The Crack and Hole detections are also analyzed with a Rule Based visual severity mechanism which classifies the amount of visibility as Low, Medium or High. These severity categories are only visual and do not offer a structural safety assessment.

- The flow of work and relationship between the major components is shown in Figure 1.



The computer vision output is then transformed into structured information, and added to the conversational part. There are six domain-specific knowledge documents in the RAG knowledge base, each stored as a LangChain Document with source metadata. The knowledge is embedded and indexed with the help of FAISS, to allow for semantic retrieval. LangChain brings together information retrieved, most recent computer vision analysis, conversation history, and current user question and feeds it to the locally running Ollama language model. This allows the application to produce explanations on the inspection and further questions based on the context, and while showing the knowledge sources retrieved.

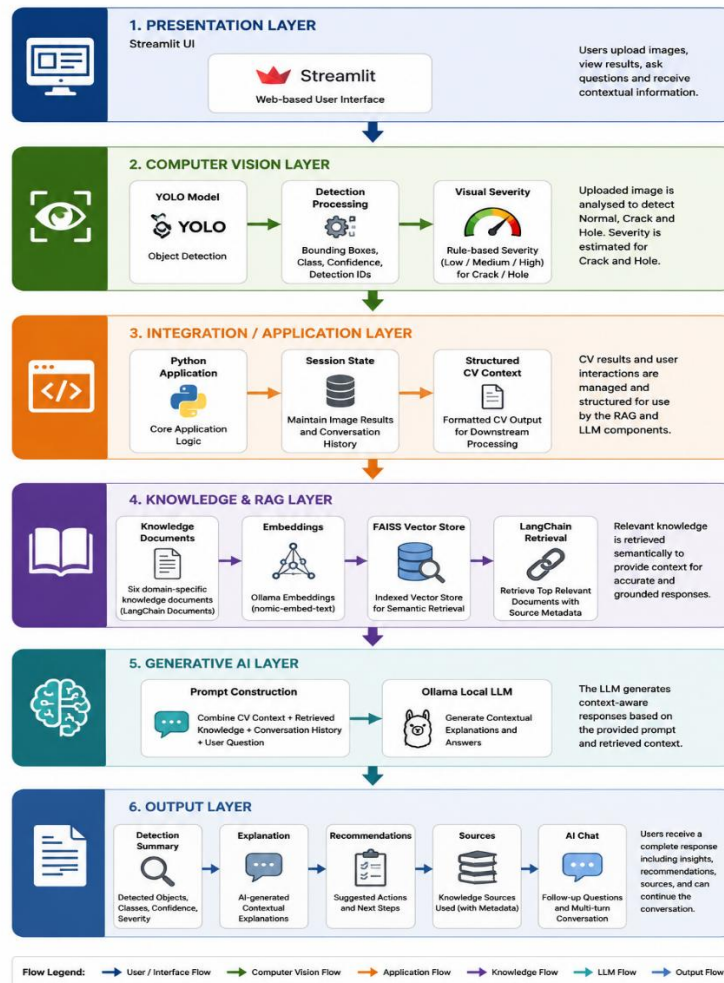
The entire prototype is presented in a Streamlit application, offering an integrated workflow for uploading images, detecting defects, tagging the severity of the found defects, reporting on context, and enabling multiple turn interaction.

2. System Design and Technology Selection

2.1 System Architecture

The proposed system is designed to be modularized and the proposed modules are computer vision, rule-based post-processing, Retrieval-Augmented Generation (RAG), and conversational AI within a Streamlit application. The architecture is designed to allow each component to complete a well-defined function and to pass well structured information to the next step in the process.

- The layered technical structure of the surface defect inspection system using AI is shown in Figure 2.



This process starts with an image of the surface being uploaded from the user interface (UI) of the Streamlit application. This image is then sent to the computer vision model built using YOLO which detects the objects and generates bounding boxes, predicted class labels, and confidence scores. Three classes are used in the model - Normal, Crack and Hole. A unique identifier is also provided for each detected object, which helps to interpret individual predictions for multiple detections in the same image.

Crack, and Hole detections are passed to a visual severity component based on rules. This component estimates the extent of the visible defect in terms of relative area and applies class-specific thresholds learnt from the training data to classify the defect into a defect severity level of Low, Medium or High. Defect severity assessment does not consider normal predictions. The resulting severity level is based on the visual extent, not structural condition or safety.

The computer vision result is structured and available to language-processing pipeline. LangChain Document objects contains domain knowledge and is indexed using a FAISS vector store. Semantically relevant information is retrieved for a user query and the latest computer vision summary, relevant conversation history and question are combined. This information is used to structure a prompt that is passed to the locally running Ollama LLM using LangChain.

The generated response is returned along with the source information via Streamlit interface. Session state keeps track of the most recent image-analysis result and conversations so that users can ask further questions, depending on the context. This architecture thus establishes a seamless flow between image analysis, contextual explanations and conversational interaction.

2.2 Technology Justification

Technologies chosen for the prototype are based on the functional requirements of the computer vision, contextual knowledge retrieval, conversational interaction, local execution, and prototype development. The core technologies covered are: YOLO for object detection, Streamlit for user interface, LangChain for orchestration, FAISS for vector-based retrieval, Ollama for local language-model execution, and Python for the primary development language.

YOLO Object Detection: The computer vision approach was chosen for the application as it required classification and localisation of the visible surface conditions. A traditional image classification model would classify an entire image into a class, but not be able to locate individual defects. It is better to go for object detection where an inspection image might have several different areas to be identified. YOLO not only gives bounding-box coordinates, but also the predicted classes and confidence score—all within a single pipeline. These output can also be down streamed as the bounding-box can be used to estimate the visual severity and the structured predictions can be passed to the conversational side. The predictive performance for the final model, however, had significant differences in performance across the classes, especially Crack and Hole. Thus, YOLO was a suitable architecture and a trained model was limited by the same performance constraints as outlined in evaluation.

Streamlit User Interface: It was decided to use Streamlit to create an accessible web-based interface as not to need the creation of a separate Frontend framework. The application needs to include uploading of images, presentation of the annotated image, inputting its severity, provision of generated reports, source information and conversational interaction. All of these functions can be directly implemented in Streamlit using Python, which decreases the complexity of implementation. The session-state flexibility is helpful to keep the most recent computer vision analysis and conversation history between user interactions. This made it possible to handle multi-turn questions without the need of a temporary conversational state database.

LangChain: LangChain has been used to coordinate the flow of retrieval, conversational context, computer vision info, and the language model. The application is not sending an open-ended question directly to the LLM, but rather creating a detailed prompt that includes the most recent CV output, domain knowledge retrieved from the knowledge base, conversation history, and the user's prompt. This offers more control over the information that is available when generating responses. Knowledge entries and their sources are represented by LangChain Document objects as well, for retrieval and source presentation.

FAISS and RAG: To serve as a vector store in the semantic retrieval, FAISS was chosen for being locally deployed and having a comparatively small knowledge base. These six documents of domain-specific knowledge are stored as LangChain Document objects and

transformed into vector embeddings. These six domain-specific knowledge documents are stored as LangChain Document objects and are converted into vector embeddings. FAISS allows for similarity-based retrieval, allowing the information passed to the LLM to be retrieved based on the semantics of the query as opposed to just exact keyword matching. It was decided to use the RAG approach to minimize the reliance on the internal knowledge of the LLM and to supply project-specific context. The application keeps source metadata for the documents retrieved, to present the knowledge source used to answer a question.

Ollama and Local LLM Execution: Ollama has been chosen to handle the execution of local language models. Local inference minimizes reliance on external cloud APIs, and does not send uploaded inspection context and conversation to a third-party LLM service while the prototype is running normally. It also eliminates API charges, which are incurred per call, in development and demonstration. The compromise is that performance will be contingent on the hardware that is available locally, and local hardware might not be as efficient as larger hardware-based models hosted in the cloud, which may generate quality responses or yield lower generation scores. The application thus does not claim to be a definitive judgment on the structure of the text but rather provides an explanatory context and a chat that helps the user.

Python and Supporting Libraries: Python was selected as the programming language for ease of common development environment for the computer vision, retrieval, LLM, data-processing and interface components. The libraries available for YOLO, image processing, LangChain and FAISS were integrated with no need to add multiple programming languages or service layers, while the Streamlit library was used to create the Streamlit app. This made the prototype development, debugging and deployment simplified.

Scalability and Cost Considerations: The current architecture is suitable for a coursework prototype due to the possibility of the knowledge base, model inference, FAISS retrieval, and LLM execution running on the local machine. This ensures that direct service expenses remain low, and deployment is easy. When there are a large number of users or knowledge collections required to be served at the same time, however, the local execution restricts the scalability. The separation of computer vision inference, vector storage, and LLM generation into distinct services and the utilization of persistent storage for user sessions and inspection records offer a potential production-scale implementation. Hence, the chosen technologies offer a realistic compromise of functionality, development time, cost and the needs of the current prototype with a clear path towards future scaling.

3 Computer Vision Model Development

3.1 Dataset Description

This computer vision part was built on a custom dataset consisting of surface images, which has been created for three classes: Normal, Crack, Hole. The Normal class points to the areas of the surface that do not have a marked defect, whereas the Crack and Hole classes indicate

two types of surface degradation that can be seen. The three classes can be used not just to detect defective areas, but also to separate them from the visually normal areas of the surface.

The data set includes images of various surface texture, lighting, defect size, orientation, background appearance and image resolution. This variation is significant as it is not the same conditions that occur during a real inspection that produce surface defects. The thin linear defects and more obvious surface cracks are crack examples, while the localised areas of surface loss or damage are Hole examples. Normal images are used as negative examples to help the model distinguish between defects and normal surface texture.

Images for object detection were accompanied by annotations describing the class and location of objects, in the form of YOLO. The annotations are drawn as a series of normalised coordinates bounding boxes. A training, validation and test set were then randomly selected from the prepared dataset to allow for model learning, stage validation and a final independent test.

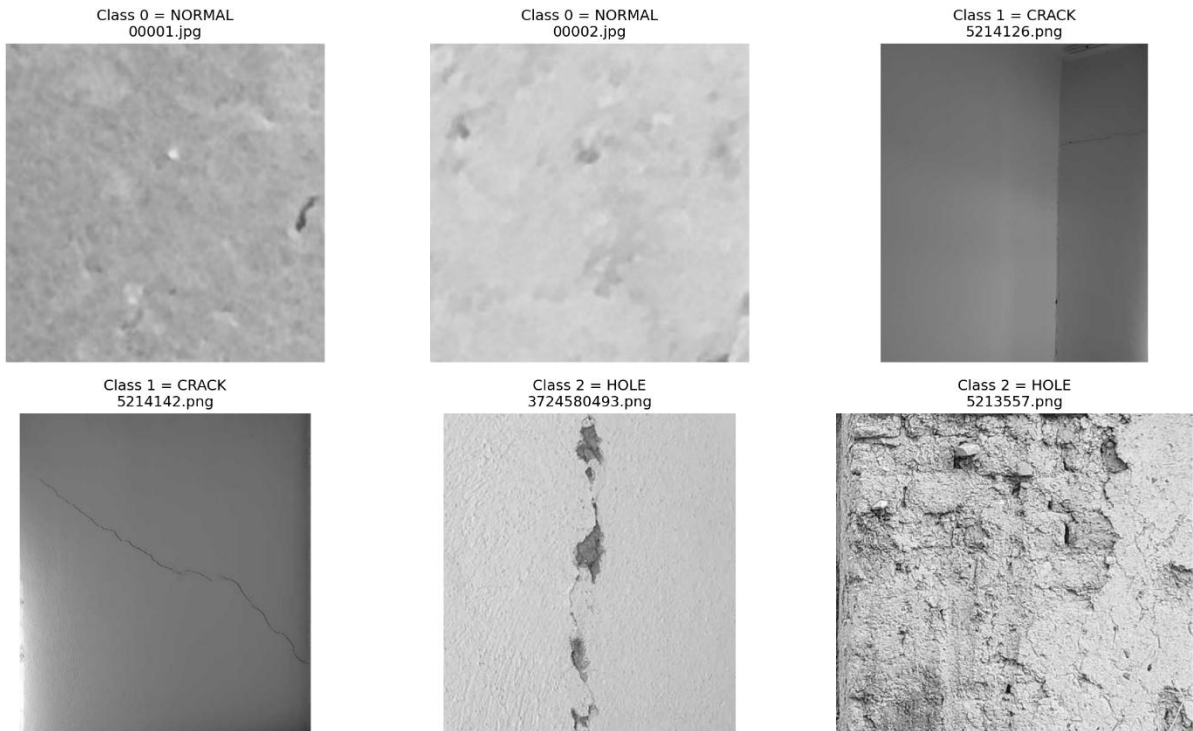
3.2 Data Collection and Class Verification

The format of the data set and the class labels were checked before training the model to make sure that the images and labels were properly aligned with the appropriate classes. The data was split into training, validation and test sets. These independent subsets are crucial since the training set is used to learn the parameters, the validation set is for performance monitoring during the development phase, and the test set is reserved to test the performance on unseen data after the development process is complete.

Visual class-verification procedure performed following the preparation of the datasets. The representative samples were chosen from each subset, and presented alongside their class labels. This allowed a pre-evaluation of the consistency of samples labeled Normal, Crack, and Hole with the nominal class definitions prior to the model training and evaluation.

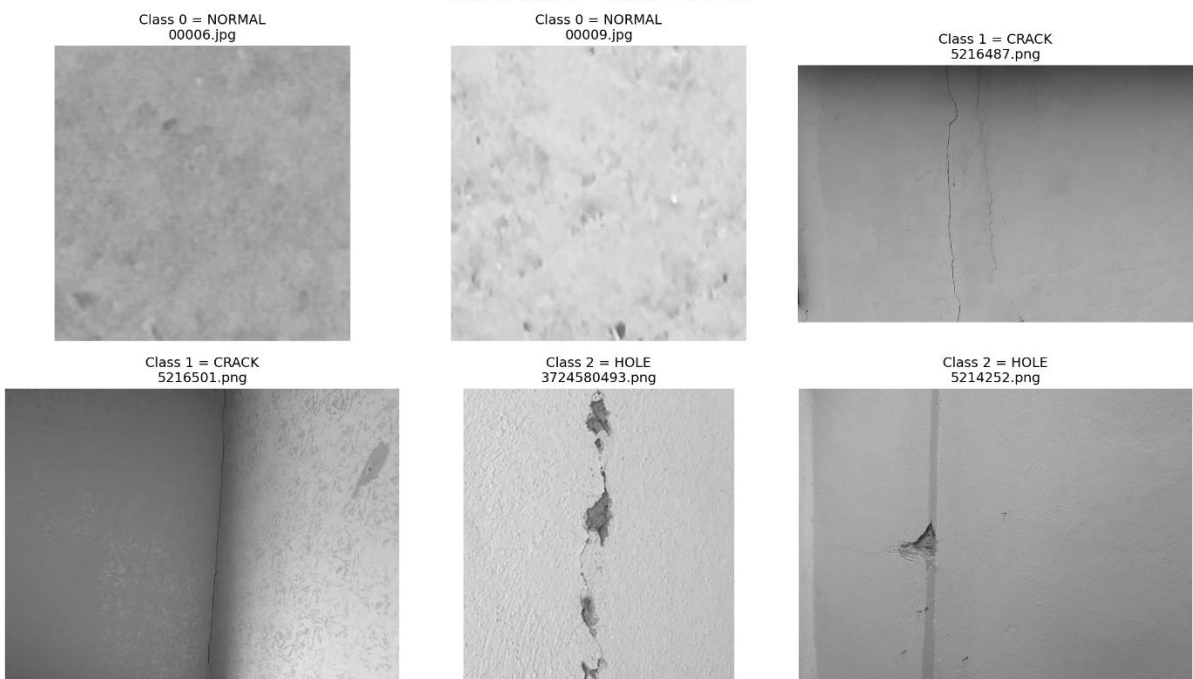
- Sample images of classes used for verification in the training subset (Figure 3).

TRAIN DATASET - CLASS CHECK

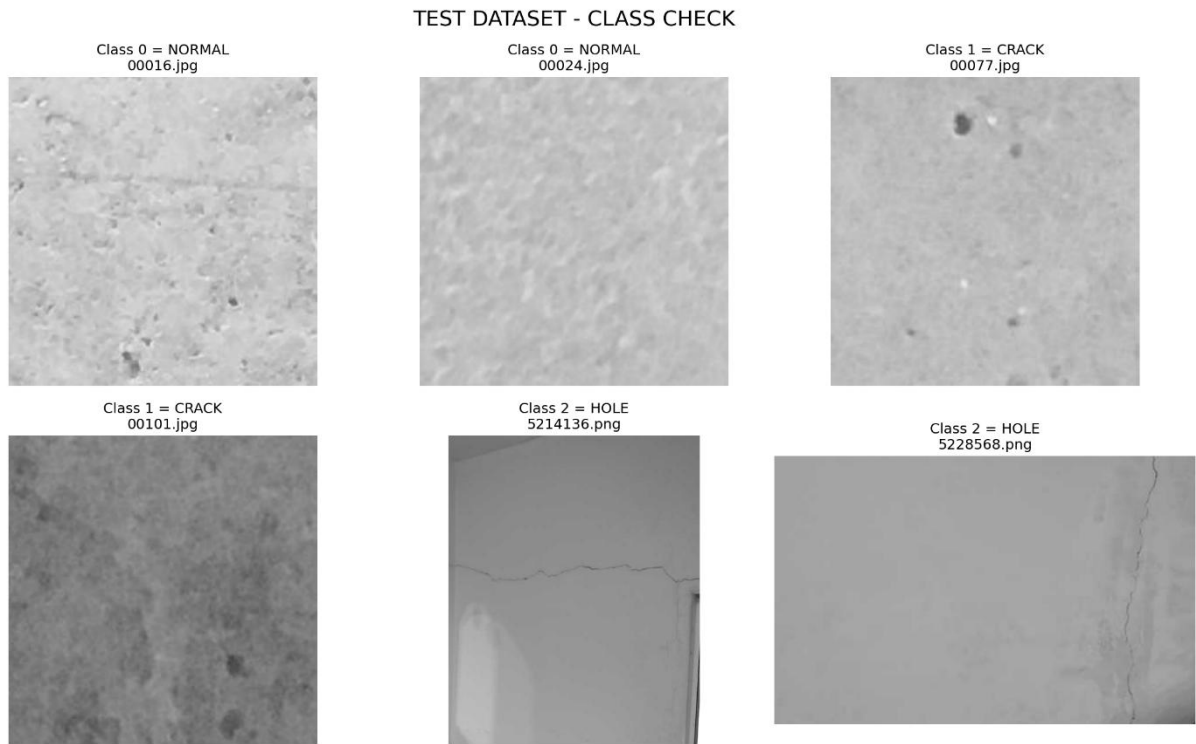


- Representative class-verification samples of the validation data set. (Figure 4)

VAL DATASET - CLASS CHECK



- Sample from the test set for each class for verification. (Figure 5)



Figures 3 - 5 show the representative class-check samples of the training, validation and test sets. During the inspection, the organisers ensured that all three target classes were evenly distributed amongst the splits of the prepared dataset. There is also some variation in the data visually demonstrated in the examples. In Crack instances, the crack length, orientation, contrast and surrounding texture are different, Hole examples are different in shape and visible surface loss. Similar surface texture and surface irregularities are also found in normal samples and could be mistaken for weak defect patterns. This variability makes the detection task more challenging, but gives a more realistic ground for assessing the model's capacity to differentiate between real defect and surface characteristics that are not. This variability complicates the detection task but gives a more realistic basis for assessing the model's capacity to differentiate real defect from surface characteristics not defect.

3.3 Data Pre-processing and Annotation

In accordance with the YOLO object detection model, the dataset was prepared as a set of labels containing a class identifier and normalised bounding box (centre position, width, and height of the target region) in the corresponding normalised bounding-boxes. Normal, Crack, and Hole classes were assigned to the data set and models.

Images are fed into the YOLO training pipeline, where it performs the image transformations required for the model input and keeps the relationship between images and bounding box annotations intact. Standardised YOLO annotations help to maintain the proportionality of labels when images are resized for training.

Particular attention was paid to the consistency of classes since an incorrect class index mapping could cause seemingly valid predictions to be interpreted using the wrong semantic label. This was particularly important when comparing the outputs of different stages of the

project. Therefore, the class names of model outputs were checked before the final evaluation and integration of the system results.

Thus, the preprocessing stage of the image was not only about preparing the images for model training but also about keeping the image files, annotations, dataset splits, and class labels consistent. This was an important part of the input data used in subsequent YOLO training and evaluation.

3.4 YOLO Model Development and Training

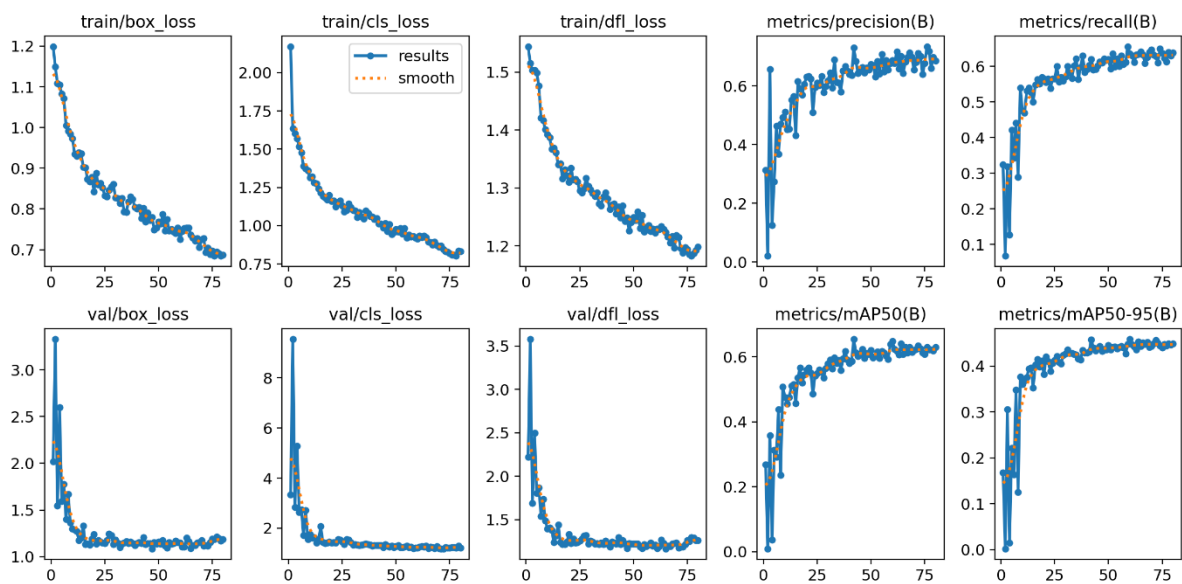
This course covers how to build and train a YOLO model.

The surface inspection problem was chosen to be a 3-class problem and a YOLO-based object detector was selected and fine-tuned for it. Transfer learning was employed, instead of training a detection network using randomly initialised parameters. This allows for previously learned visual features to be transferred to the target dataset, and is especially applicable when there is a comparatively small amount of task-specific data available, which is the case with many large-scale datasets needed to train deep neural networks from scratch.

Images along with the corresponding YOLO annotations in the training subset were provided to the model during training. The validation subset is assessed in the training process to determine the level of transfer of the learned representations from the subset used to optimise the parameters to the validation subset. During the training process, there arose some loss functions like bounding-box loss, classification loss, distribution focal loss, etc. Meanwhile, validation metrics like precision, recall, $mAP@0.5$, $mAP@0.5-0.95$ were tracked throughout the epochs.

These training curves display a consistent decrease in the main training losses with the progression of learning. The values of training box loss, classification loss, and distribution focal loss decreases as the number of epochs, showing that the model gradually improves its localization and classification of the labelled objects in the training data. The validation losses are significantly lower in the early part of the season and then become relatively constant. The precision, recall and mAP also increase through training and eventually begin to level off.

- Figure 6: YOLO training and validation loss and performance curves across the training epochs.



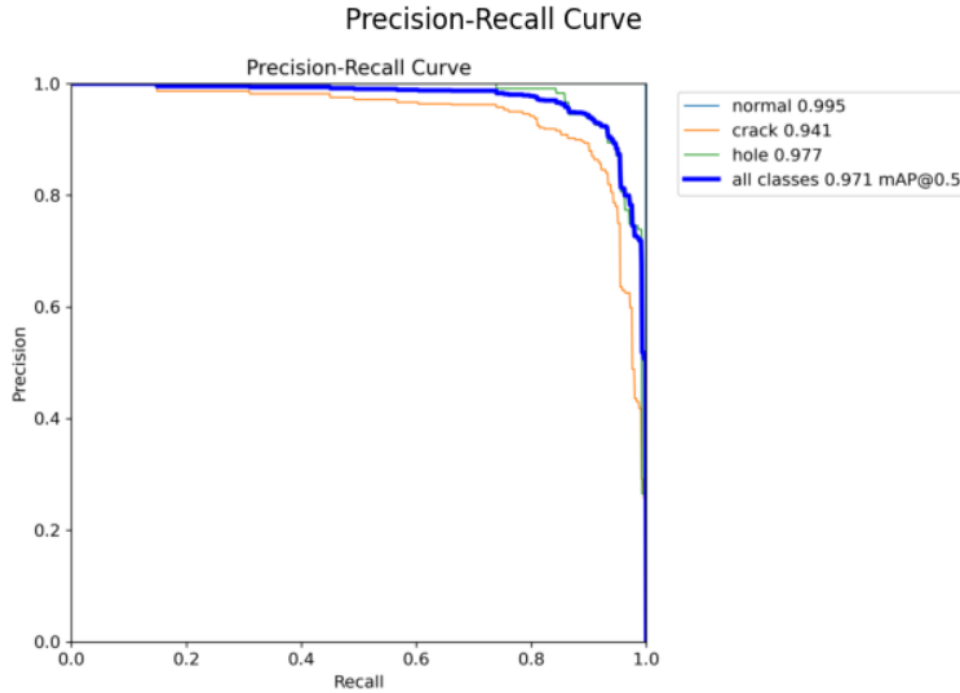
The curves provide development-stage evidence of model convergence and should not be interpreted as final held-out test-set performance.

These curves show that the optimisation process converged and useful visual properties were learnt from the prepared data set. Training and validation metrics, however, should not be taken as the ultimate representative of the performance of the deployed model when used independently. These are indicators for the development stage created during the training run of YOLO. As a result, final performance is reported separately, and evaluated according to held-out test.

3.5 Model Outputs and Training-Stage Analysis

Apart from the epoch-based Training Curves, the YOLO training process created data for Diagnostic outputs consist of class-distribution information, confusion matrices, precision–recall curves, precision–confidence curves, recall–confidence curves, F1–confidence curves, labelled training batches and the prediction samples for validation. These outputs were used during model development to check the behaviour of the model and to determine how well the detection performance differed between the different classes being targeted.

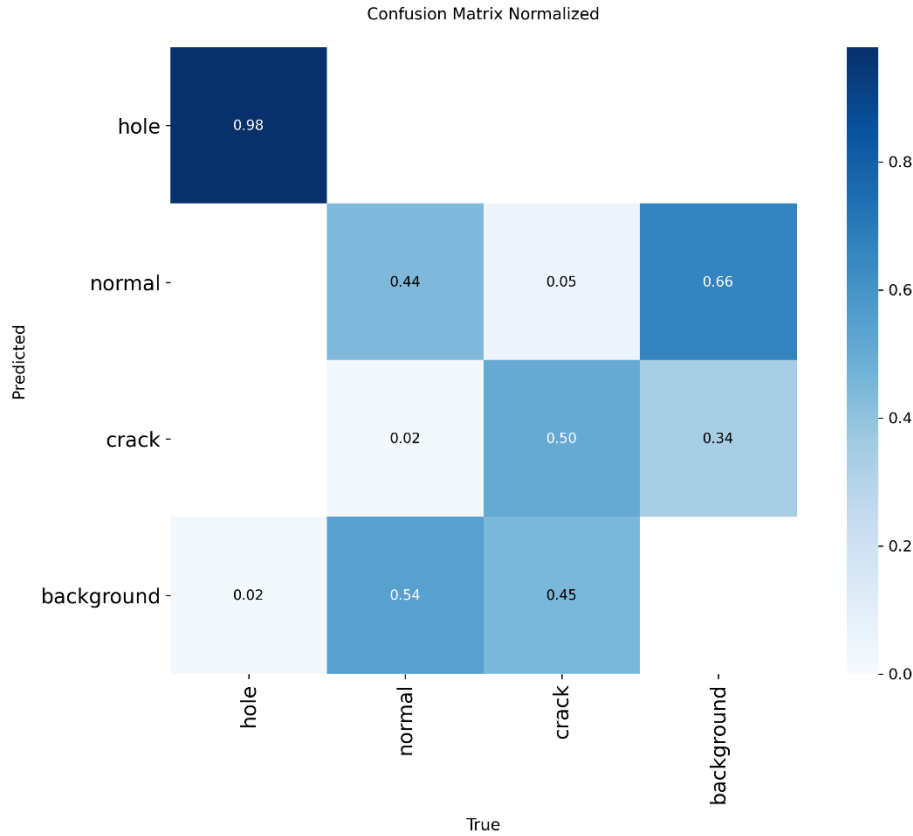
- Figure 7: Class wise Precision Recall curves generated during the YOLO training and validation process.



Diagnostic results suggest that performance for the model is not overall equitably distributed in the classes. The training-run diagnostic outputs for the Hole class are significantly better than the Normal and Crack classes in particular. For instance, the precision–recall curve has a significantly larger area for Hole than for the other classes. This indicates that the visual properties embodied by the class Hole were easier to learn uniformly by the trained detector, while the other classes were more ambiguous, in part due to the presence of Crack.

The confusion-matrix and the confidence-based diagnostic plots also show false positives, missed detections and confusion among background regions. These observations can help to understand the constraints of the model, and identify potential improvements to the data quality, consistency of the annotations, and class distribution, as well as training the model.

- Figure 8: Normalized confusion matrix obtained during the development-stage evaluation of the YOLO model.



Importantly, these figures are from the YOLO training and validation run and thus not considered the final test-set evaluation and should be used to serve as a diagnostic development piece of evidence. In the evaluation section, the held-out test data set is separately evaluated to report final performance values.

Class-check samples are shown in Figures 3–5 that are representative from the training, validation, and test datasets. A visual inspection revealed that all three target classes (Normal, Crack and Hole) were present in each of the splits prepared in the dataset. Examples also show some variation in the visual characteristics within each class. The differences in length, orientation, contrast, and surface texture of the surrounding surface of the crack instances are also different, as are the differences in shape and amount of visible surface loss of the Hole examples. Natural surface textures and irregularities can also occur in normal samples that can visually appear as weak defect patterns. This variability makes the detection task more difficult as it will be more complex to differentiate the true defects from the non-defective surface properties at various visual conditions. Rather, such variation offers a more realistic measure for testing the model's generalisation properties with respect to different surface appearances.

4 Model Evaluation and Performance Analysis

4.1 Evaluation Approach

After developing the model, the model was tested using held-out test set and not the images that were used for model training and model development validation. The aim of this evaluation was to evaluate the trained detector's ability to generalise well to unseen images of surfaces with the three target classes: Normal, Crack and Hole.

The system performance was evaluated with common object-detection metrics including precision, recall, F1-score and mean Average Precision (mAP). Precision quantifies the percentage of the model's predictions that are accurate, and recall quantifies the percentage of objects in the ground truth that are predicted by the model. When both false-positive and missed detections are relevant, the F1-score is a combined measure of precision and recall. The Mean Average Precision takes into account the quality of the classification and the quality of the localisation for the classes evaluated and gives an overall picture of the object-detection performance.

The test evaluation was carried out without relying on the diagnostic results obtained while training YOLO. This distinction has been preserved to observe for the convergence and the development of the model, but it is not to be understood as evidence of final generalisation performance. In the following, therefore, all the numerical results are limited to the test evaluation at the end.

4.2 Training and Validation Performance

The learning trend was stable during the training and validation stage. Actually, the convergence behavior shown in training suggests that the YOLO model gradually acquired the meaningful visual features for the three classifications (Normal, Crack and Hole). As mentioned in the Figure 6, the training loss and validation loss both decreased steadily and the precision, recall and mAP metrics rose with successive epochs. The obtained results indicate that the model training process converged properly and the learnt parameters were appropriate to be evaluated later. These diagnostic results are at the model development phase only, however, and should not be viewed as the final held-out test results.

The training and validation convergence shown above in Figure 6 shows that the optimisation losses converge over time and that precision, recall and mean Average Precision (mAP) gradually improve. The above observations suggest the model converged well in training and acquired discriminative visual representation of the target surface conditions.

4.3 Analysis of precision, recall and F1-score.

Precision and recall provide complementary information about the behaviour of the detector. The higher the precision the more correct predictions made by the model match the objects labelled as "correct" and the higher the recall the more objects in the labelled set were

detected. In a surface-inspection application, both measures are important. Low precision can lead to false defect alerts and low recall can lead to surface defects going undetected.

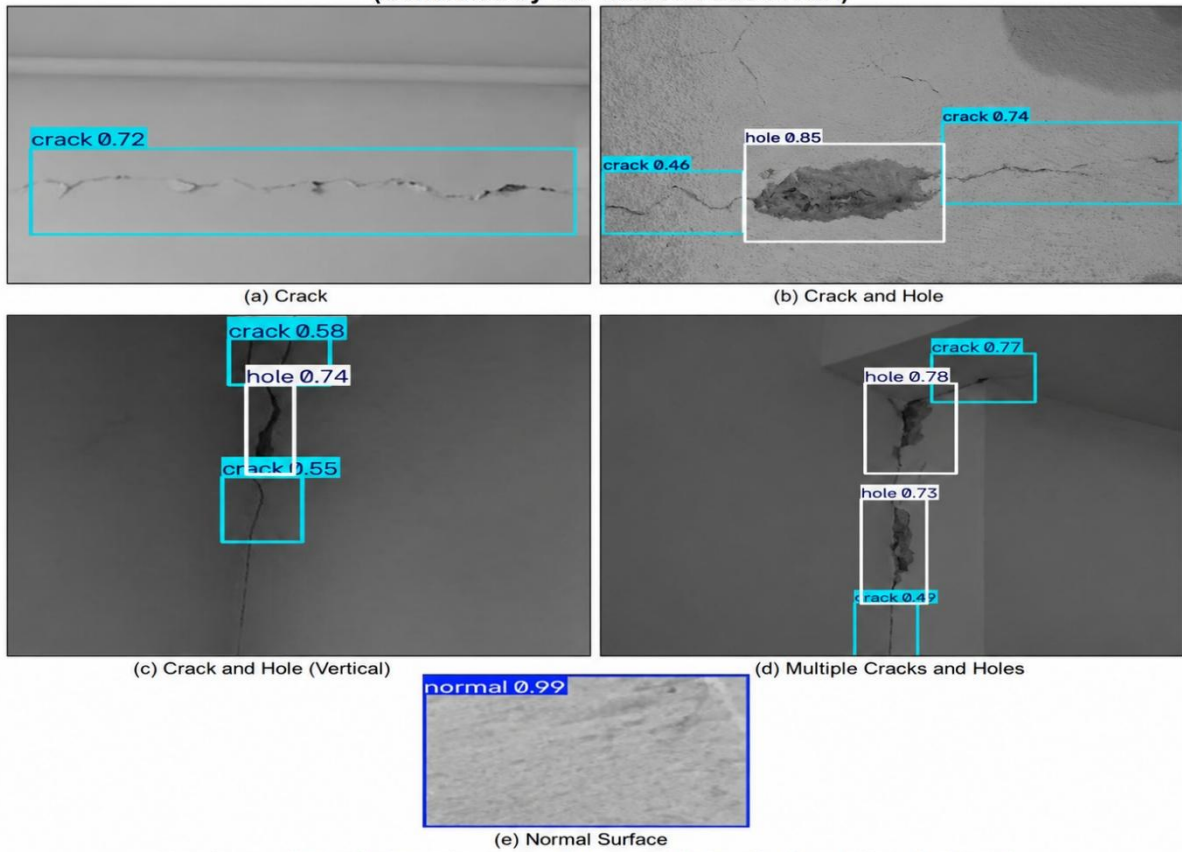
Both of these measures are balanced by the F1-score. Test results show the reliability varies among the target classes. This is consistent with the fact that during dataset verification, the datasets are observed to be visually complex for Crack and Normal examples. Thin cracks can be just a small portion of an image and can sometimes lack contrast to the rest of the image surface. Likewise, with Normal images there may be natural texture that looks like weak crack or damage patterns.

This is because the improvement in performance is more pronounced for Hole, which indicates that the model can pick up on more distinctive, and localised, regions of surface loss. However, the distinctions between classes suggest that the prototype should not be seen as being equally reliable for each category of defect. This is taken into account when interpreting the results to the user and is also mentioned in the recommendations for further development of the model.

4.4 Detection Examples and Qualitative Evaluation

Qualitative assessment of predictions on the unseen test images was complemented by quantitative metrics. Representative predictions were investigated to check the meaningfulness of the bounding boxes predicted on visible surface areas, and the visual reasonableness of the assigned class labels. This is significant because this kind of inspection is necessary to provide a complete picture of the behavior of a detector on individual images which is not fully captured by numerical metrics.

**Figure 9 – Representative Prediction Results
(Generated by the Trained YOLO Model)**



These images show prediction results on unseen test images. The bounding boxes indicate the detected objects with their predicted class labels and confidence scores.

- Figure 9. Prediction result for each image using the trained YOLO model.

The detected surface conditions are shown with bounding boxes, the predicted class labels and scores. The examples show the successful detection of the Normal, Crack and Hole surface conditions under various visual conditions.

The examples show that the model can localise surface defects that are easily visible and provide the class and confidence level information. The number of holes detected is usually more conspicuous, and there is a correspondence between the number of holes detected and the number of quantitative hits. The detection of cracks is more difficult if they are thin, light, partially hidden, or resemble natural edges and texture on the surface.

A qualitative assessment has also revealed examples of instances where the qualitative assessment could fail to identify some weak defects, or where the qualitative evaluation could be triggered by a visual pattern that is not a defect. The examples are significant for their role of illustrating the prototype's restrictions and not just the successful detections. The system thus provides confidence information, and considers its outputs as decision support information not as a structural assessment.

4.5 Error Analysis and Model Limitations

Test results were analysed to determine some of the factors that affect the reliability of detection. A critical problem is the similarity of visual appearance between weak and normal surface properties. Thin cracks can appear as joints, shadows, edges or variations in texture; some Normal surfaces may have irregular patterns, which may create confusing visual features. The appearance of the target classes may be further influenced by variations in illumination, image quality, viewing distance, defect size and material characteristics of the surfaces.

The behaviour of the model depends on the characteristics of the data. The detector may learn the class with more consistent examples better than the other class, if there are more examples of that class. There may therefore be more variation between individuals within Crack and Normal class groups than between individuals across those two groups, which can lead to differences in performance when assessed. More examples of varying and properly annotated examples would be needed to enhance generalisation to a wider variety of real inspection contexts.

A key challenge in the development process was the class mapping consistency, from data preparation to the outputs of YOLO, training diagnostics, to the integrated application. Some training run numbers were a result of class ordering from the development pipeline and thus could not be used automatically as final test numbers. In order to avoid giving misleading figures as training and validation data remained as evidence of the development stage and the final performance was tested separately using the held-out test set and the class mapping that had been verified.

It is important to realize that the current model is not intended to replace a professional structural inspection but rather is a working prototype only. The visual severity categories and explanations generated are a source of additional contextual support, but not a source of structural safety, defect depth, material integrity, engineering risk.

4.6 Evaluation Summary

Overall, the developed YOLO model shows significant performance in identifying and localising the three target surface categories, but the performance varies slightly between the different categories. Crack and Normal are more difficult to classify than Hole due to higher visual ambiguity and visual variability. The use of quantitative test metrics and qualitative prediction inspection gives a more comprehensive assessment than either alone.

The evaluation also emphasizes the need for distinction between training and held-out test diagnostics and evaluation. This way, generalisations about the model are made with unseen data and not on observations at the development stage. The results indicate the usefulness of the model in the prototype as an assistive inspection component and highlight the potential for further advances with increased use of more, better annotated, more diverse classes and more extensive model development.

5 Conclusion and Future Directions

5.1 Conclusion

The goal of this project was to create an intelligent prototype that could identify visible defects on a surface and generate contextually relevant explanations with the incorporation of computer vision and Retrieval-Augmented Generation (RAG). The developed system combines a YOLO-based object detection model, a Streamlit web interface, a FAISS vector database, LangChain retrieval pipeline and a locally deployed Ollama large language model to create an end-to-end inspection workflow. In contrast to the detection outputs, the prototype also offers context information and conversational support, helping the user to understand the detected surface conditions.

The computer vision part was able to detect the three classes (Normal, Crack and Hole) and output the bounding boxes, confidence scores, and visual severity information for each object detected. The development-stage training diagnostics yielded stable convergence during the learning process, and the model was able to recognize the desired surface conditions in unseen images, as ascertained by qualitative evaluation. The performance was found to be different for different classes, but the overall behaviour of the detector suggests that the trained model was able to capture meaningful visual representations from the prepared dataset.

Incorporating the Retrieval-Augmented Generation framework greatly enhances the capabilities of a traditional object detection system. The structured computer vision output is then augmented with pertinent domain knowledge from the vector database, which is passed to the language model rather than presenting just numbers. The prototype can therefore produce a natural language explanation, inspection recommendation and follow-up conversation response that are directly related to the defects detected. This enhances the understandability of the inspection process and for users who would like more guidance than just the prediction.

The developed prototype shows that computer vision and retrieval-based language models are capable of successfully being combined into one intelligent inspection system. The prototype is designed for supporting professional engineering judgment and decision making process, but the results obtained show that the proposed approach is practically viable for supporting and assisting surface defect inspection and make the access to AI assisted structural assessment more accessible.

5.2 Limitations

The obtained results from the developed prototype show promising performance, but a few limitations should be taken into consideration when interpreting the obtained results. Variation in the quality of the object detection is dependent on the diversity and quality of the data used for training. The dataset used for preparation is limited and the model trained is limited to the conditions used in the data set, so the generalisation is limited to all real-world inspection scenarios.

Some natural limitations are caused by the similarity of the visualization of some surface features. The difference between defective and non-defective regions is more difficult when there are natural surface textures, fine cracks, shadows, stains and changes of illumination. Therefore, there will be momentary difficulty in detection, especially if the defects are small and/or partially visible or difficult to detect under lighting conditions.

The quality of document collection in the Retrieval-Augmented Generation pipeline also plays a key role in the knowledge generation part. The explanations produced are limited to the indexed documents that exist in the vector database, so if there isn't enough or no engineering knowledge related to the documents, the responses may be less informative. Moreover, the locally deployed language model can offer efficient offline inference without having the knowledge coverage of the larger cloud-based language models.

Lastly, the proposed system could be considered as a prototype of an intelligent inspection support system, which should not be used as a certified engineering assessment system. Predictions, severity estimations and AI explanations generated are for the purposes of supporting inspection activity and should be always used in consideration with professional engineering knowledge and field experience.

5.3 Future Work

The following improvements can be implemented in the next proposed prototype to better the computer vision and intelligent assistance parts of the prototype. An extension of this would be to expand the training set by capturing more images with annotations, under varying conditions of lighting, environmental conditions and camera viewpoints. Other defect types like spalling, corrosion, water leakage and surface deformation could be added to enhance the system applicability to a wider range inspection tasks of structures.

Another way to enhance the RA-G process is to expand the engineering knowledge base. Incorporation of technical manuals, engineering reference documents, maintenance standards and guidelines for structural inspections would enhance the relevance and quality of explanations/recommendations generated. Other locally deployable language models, which are more sophisticated, may also be used to facilitate a richer kind of reasoning, in addition to being offline.

In the future, development can also be done to improve the usability of the application itself. Being able to view the inspection in real time, be able to view and use it on mobile devices, generate automatic inspection reports, sync to the cloud, and access the inspection history in the past would make a huge difference in the practical use of the system in the field by engineers and maintenance staff.

In general, the proposed prototype provides a solid baseline of future intelligent structural inspection systems. Advances in computer vision, Retrieval-Augmented Generation and large language models could result in more accurate, scalable and practically useful applications that provide support for infrastructure maintenance and engineering decisions in the future.

REFERENCES

LangChain. (2025). *LangChain documentation*. <https://python.langchain.com/docs/>

Meta AI. (2025). *FAISS: Facebook AI Similarity Search*.
<https://github.com/facebookresearch/faiss>

Ollama. (2025). *Ollama documentation*. <https://ollama.com/>

Python Software Foundation. (2025). *Python documentation*. <https://www.python.org/>

Streamlit Inc. (2025). *Streamlit documentation*. <https://docs.streamlit.io/>

Ultralytics. (2025). *Ultralytics YOLO documentation*. <https://docs.ultralytics.com/>

YOLOv8 (Official Documentation) [Ultralytics YOLOv8 Documentation](#)

Retrieval-Augmented Generation (RAG) [RAG Research Paper \(NeurIPS 2020\)](#)

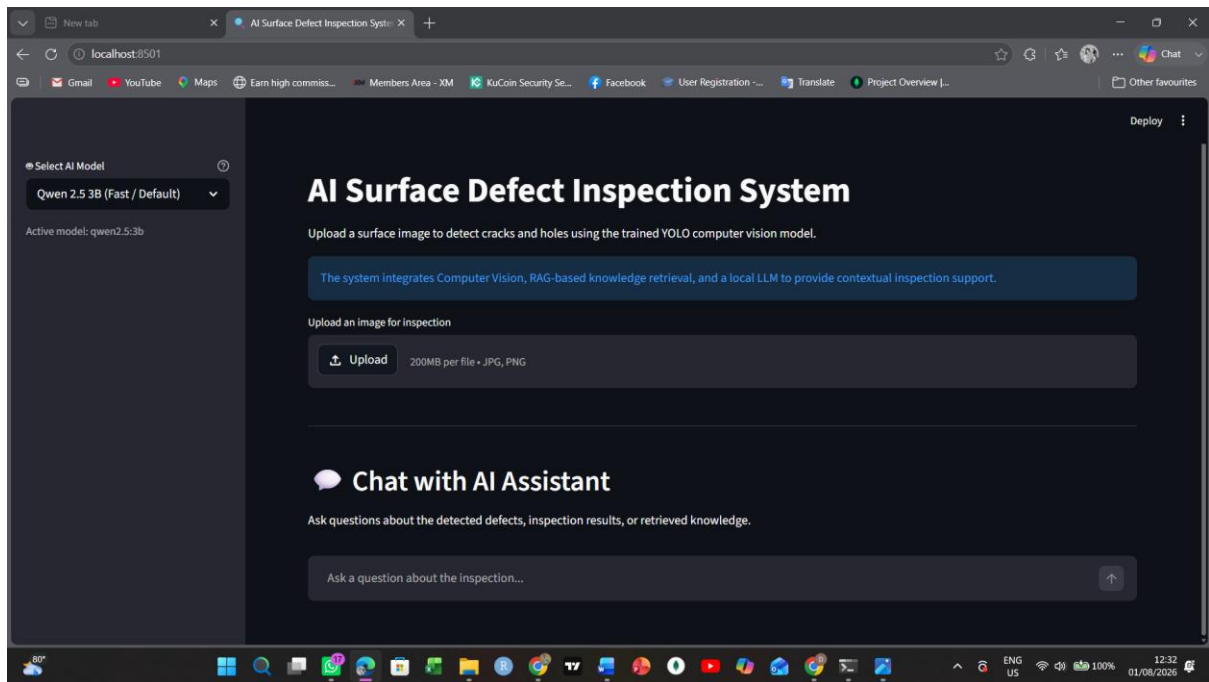
FAISS Research Paper [FAISS Paper \(IEEE\)](#)

Crack Detection using CNN [Road Crack Detection using Deep CNN \(IEEE\)](#)

Appendix Part-A

Application Interface Screenshots

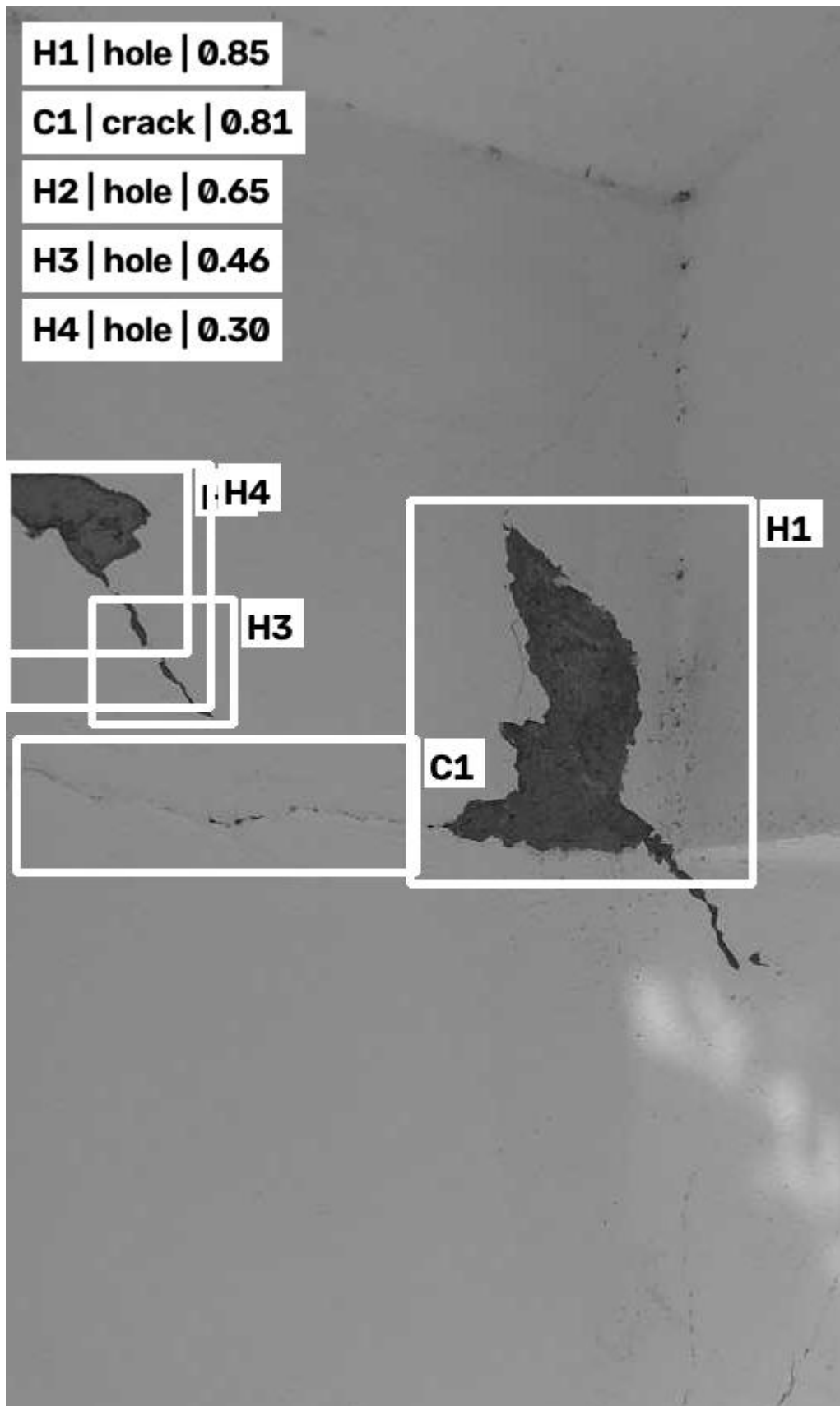
Figure A.1



- Main Streamlit interface of the proposed AI-based surface defect inspection system, including features for uploading images and analysing them using different models.

Detection Result Screen

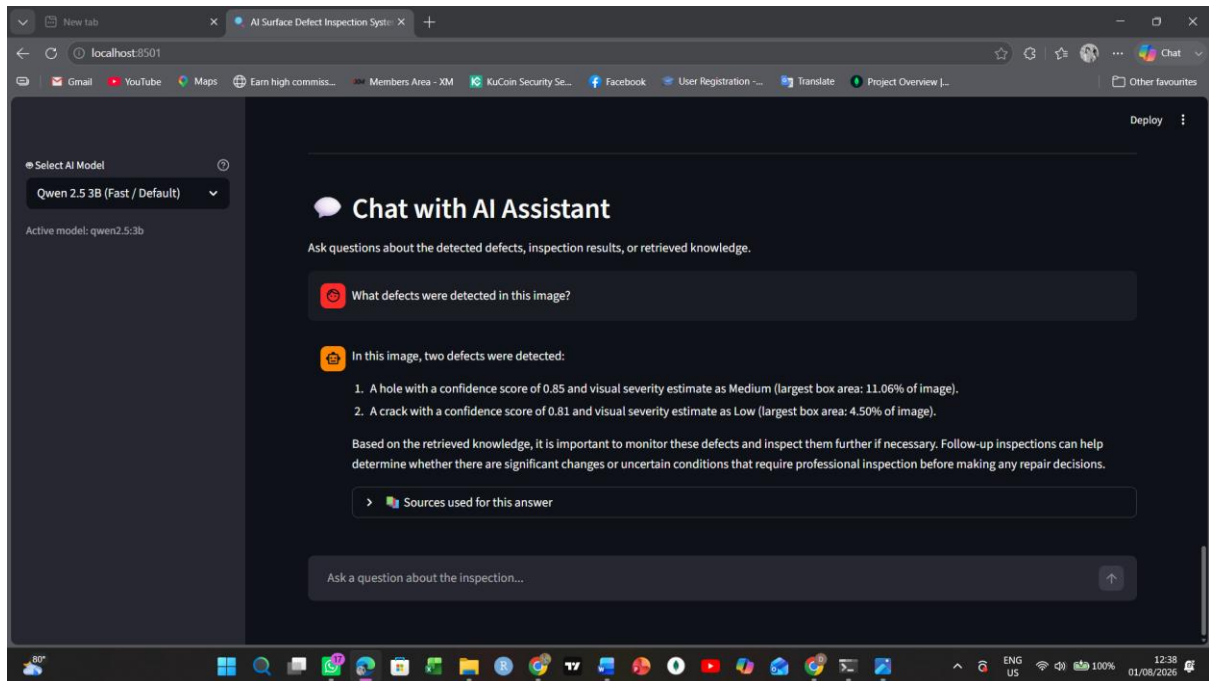
Figure A.2



- Results of the detection provided by the trained YOLO model, with predicted class label, bounding box, confidence score, detection identifier and visual severity assessment of the detected defects.

AI Chat Screen

Figure A.3



- Interactions with the integrated Retrieval-Augmented Generation (RAG) assistant, showing contextual explanations, recommendations, and source-supported responses based on the computer vision output.

Appendix Part-B

Dataset Class Distribution Verification in Jupyter Notebook

Figure B.1

```

for cls in ["train", "test"]:
    print(f"====={cls}")
    print(cls)

    labels = os.listdir(f"Crack_Hole_Normal_Dataset/labels/{cls}")

    normal = 0
    crack = 0
    hole = 0

    for file in labels:
        with open(f"Crack_Hole_Normal_Dataset/labels/{cls}/{file}") as f:
            for line in f:
                c = int(line.split()[0])

                if c == 0:
                    normal += 1
                elif c == 1:
                    crack += 1
                elif c == 2:
                    hole += 1

    print(f"Normal = {normal}")
    print(f"Crack = {crack}")
    print(f"Hole = {hole}")

=====
train
Normal = 683
Crack = 1286
Hole = 597
=====
test
Normal = 215
Crack = 306
Hole = 195

```

- An output from a Jupyter Notebook which includes the distribution of the classes in the training and test sets (Normal, Crack and Hole) prior to model training. Jupyter Notebook output that displays class distribution (Normal, Crack, and Hole) before training the model in the training and test sets.

Final YOLOv8 Model Training and Validation Output

Figure B.2

```

100 epochs completed in 35.686 hours.
Optimizer stripped from C:\Users\HP\OneDrive\Desktop\VL Applications practical\VL CW NHPT_Project\runs\detect\NHPT_Project_v2\Fine_Tuned-2\weights\last.pt, 22.5MB
Optimizer stripped from C:\Users\HP\OneDrive\Desktop\VL Applications practical\VL CW NHPT_Project\runs\detect\NHPT_Project_v2\Fine_Tuned-2\weights\best.pt, 22.5MB

Validating C:\Users\HP\OneDrive\Desktop\VL Applications practical\VL CW NHPT_Project\runs\detect\NHPT_Project_v2\Fine_Tuned-2\weights\best.pt...
Python-3.11.5 torch-2.13.0+cpu CPU (11th Gen Intel Core i7-116507 @ 2.80GHz)
Ultralytics 8.4.96 Python-3.11.5 torch-2.13.0+cpu CPU (11th Gen Intel Core i7-116507 @ 2.80GHz)
Model summary (fused): 73 layers, 11,126,745 parameters, 0 gradients, 28.4 GFLOPs

```

Class	Images	Instances	Box(P)	R	mAP50	mAP50-95
all	320	512	0.929	0.94	0.971	0.801
normal	136	136	0.982	1	0.995	0.995
crack	154	242	0.869	0.909	0.941	0.646
hole	85	134	0.935	0.91	0.977	0.762

```

Speed: 1.9ms preprocess, 223.5ms inference, 0.0ms loss, 0.6ms postprocess per image
Results saved to C:\Users\HP\OneDrive\Desktop\VL Applications practical\VL CW NHPT_Project\runs\detect\NHPT_Project_v2\Fine_Tuned-2

```

Out[36]: ultralytics.utils.metrics.DetMetrics object with attributes:

```

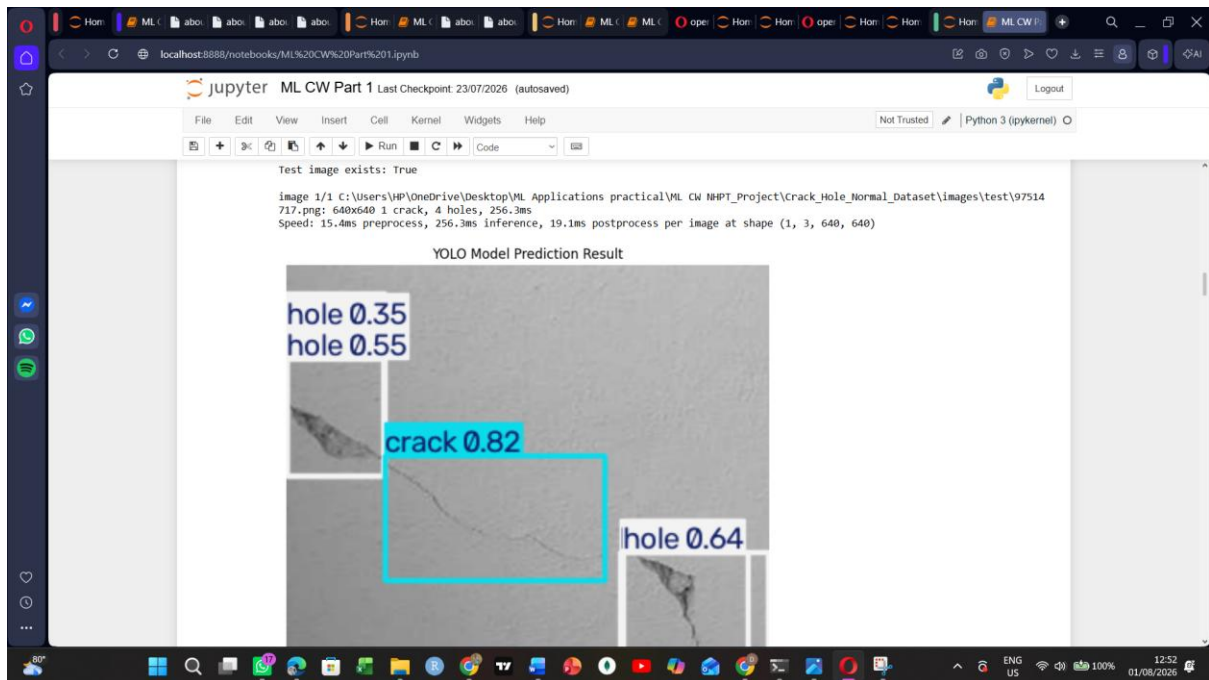
ap_class_index: array([0, 1, 2])
box: ultralytics.utils.metrics.Metric object
confusion matrix: ultralytics.utils.metrics.ConfusionMatrix object at 0x0000023103390050
curves: ['Precision-Recall(0)', 'F1-Confidence(0)', 'Precision-Confidence(0)', 'Recall-Confidence(0)']
curves results: [[array([
0, 0.001001, 0.002002, 0.003003, 0.004004, 0.005005, 0.006006, 0.007007,
0.008008, 0.009009, 0.01001, 0.011011, 0.012012, 0.013013, 0.014014, 0.015015, 0.016016, 0.017017,
0.018018, 0.019019, 0.02002, 0.021021, 0.022022, 0.023023,
0.024024, 0.025025, 0.026026, 0.027027, 0.028028, 0.029029, 0.03003, 0.031031, 0.032032,
0.033033, 0.034034, 0.035035, 0.036036, 0.037037, 0.038038, 0.039039, 0.04004, 0.041041, 0.042042,
0.043043, 0.044044, 0.045045, 0.046046, 0.047047,
0.048048, 0.049049, 0.05005, 0.051051, 0.052052, 0.053053, 0.054054, 0.055055, 0.056056,
0.057057, 0.058058, 0.059059, 0.06006, 0.061061, 0.062062, 0.063063, 0.064064, 0.065065, 0.066066,
0.067067, 0.068068, 0.069069, 0.07007, 0.071071,
0.072072, 0.073073, 0.074074, 0.075075, 0.076076, 0.077077, 0.078078, 0.079079, 0.08008

```

- YOLOv8 model training and validation final output: The final output of the training and validation process of the YOLOv8 model, including the process of training and validation, and the overall detection performance.

Sample Prediction Generated by the Trained YOLO Model

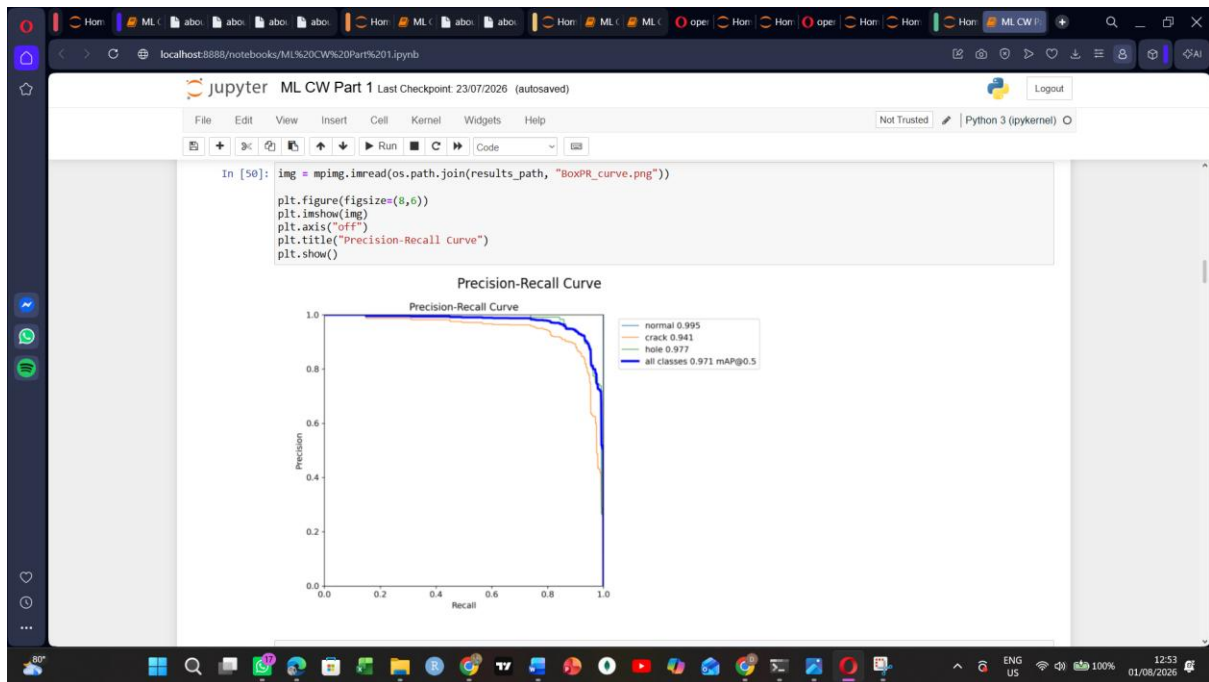
Figure B.3



- The trained YOLOv8 model gives detection of surface defects, class prediction, confidence, and bounding boxes for the samples.

Precision–Recall Curve Generated During Model Evaluation

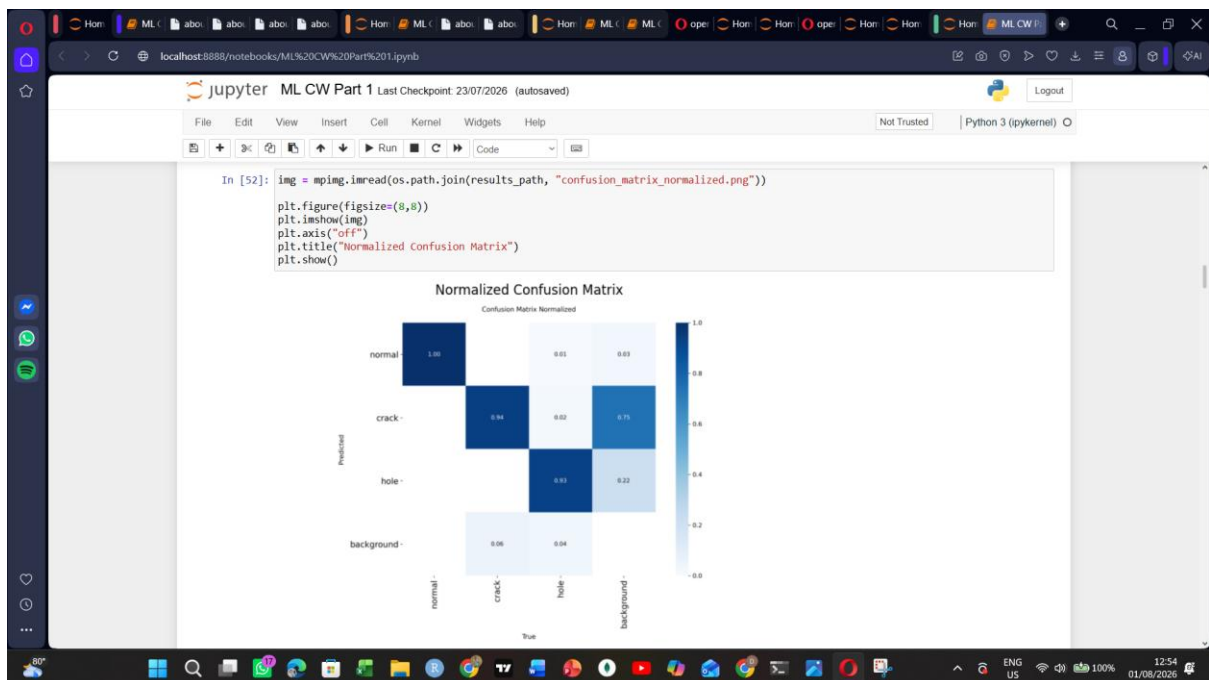
Figure B.4



- The detection performance of the trained YOLOv8 model with respect to the three target classes during evaluation is shown by the Precision-Recall (PR) curve.

Normalized Confusion Matrix of the Trained Model

Figure B.5



- Normalized confusion matrix of the trained YOLOv8 model's performance on the test set.

